

Recurrent Neural Network (RNN) Long Short Term Memory (LSTM)

Dr. Grantham Pang

What kind of problems does it handle ?

1. Sometimes, the meaning of a sentence depends on the words that came before it.

Example: “The room is **light**.” light means brightness

“The bag is **light**.” light means not heavy

Recurrent Neural Networks (RNNs) become useful in understanding the **context** of a sentence.

2. The word order in a sentence and its meanings.

Example: “The dog bit the man.”

“The man bit the dog.”

3. Time series data of stock price

20 values

$x_{t-19} \quad x_{t-18} \quad \dots \quad x_t \quad \longrightarrow \text{Predict next value}_2$

What is an RNN ?

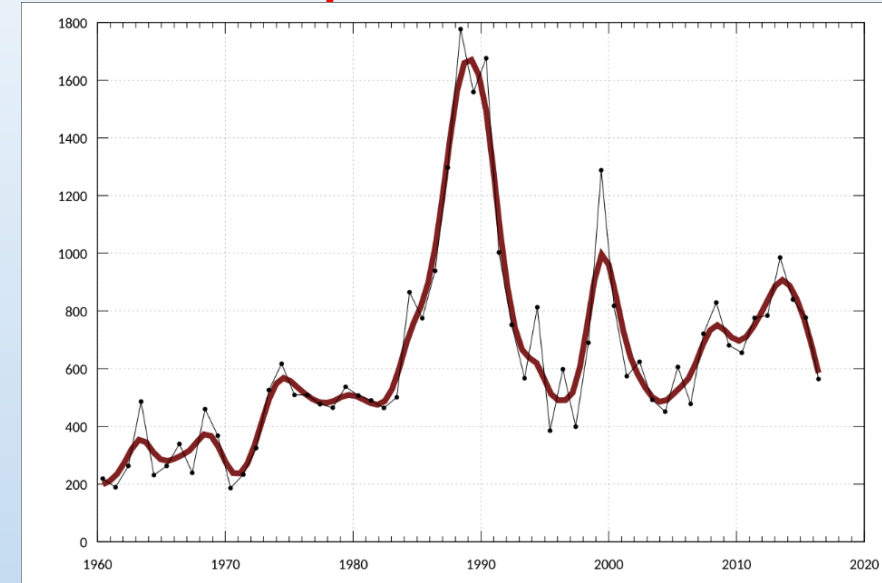
It is a type of neural network designed to work with **sequential data**.

Sequential data is data where

the **order of information matters**, such as:

Text and sentences, Speech, Time-series data,

Stock prices, Sensor readings. **Note that the nature of data can be very different.**



We need a model that can look at the current input while also remembering the past.

Language — words come in a specific order

Audio — sounds occur one after another

Stock prices — today's value is related to previous values

DNA — the order of nucleotides carries information

Unlike a normal neural network, an RNN can use information from previous steps while processing the current input.

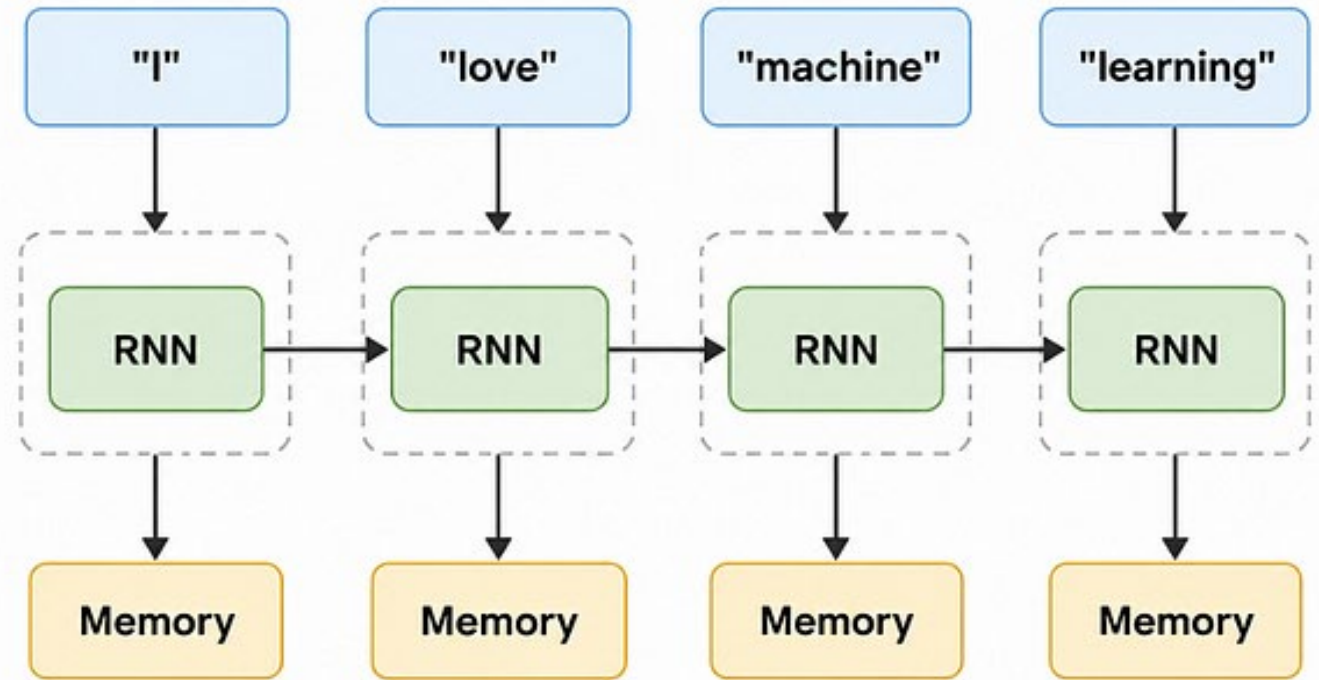
Think of an RNN as having a memory.

Example: On speech processing, the model does not look at every word independently. Instead, it carries some information from previous words to the next word.

e.g. I → love → machine → learning

The RNN processes the words one by one and keeps updating its memory.

Input words



How Does an RNN Work?

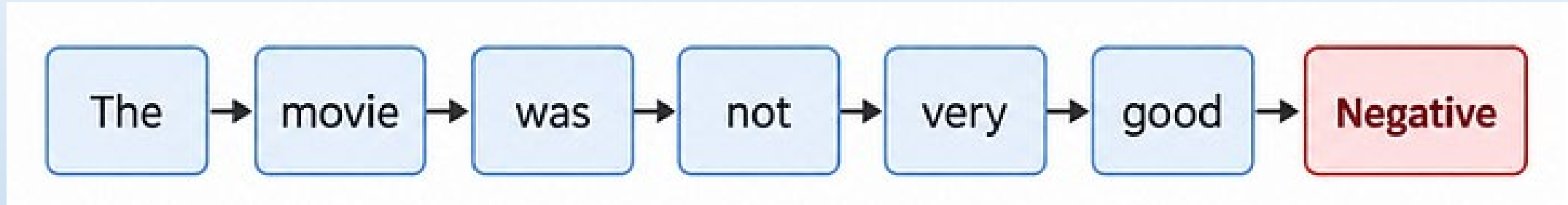
At every time step, the RNN

1. Takes the current input
2. Looks at its previous hidden state
3. Combines them to create a new hidden state
4. Uses that information to produce an output

An example on Sentiment Analysis:

Consider: “The movie was not very good.”

An RNN reads the sentence step by step:



The word “not” changes the meaning of “good”.

Because an RNN processes words in sequence and maintains a hidden state, it can use earlier information to understand the later words.

This makes RNNs useful for tasks such as:

Sentiment analysis; Text classification; Language modeling;
Speech recognition; Text generation; Machine translation.

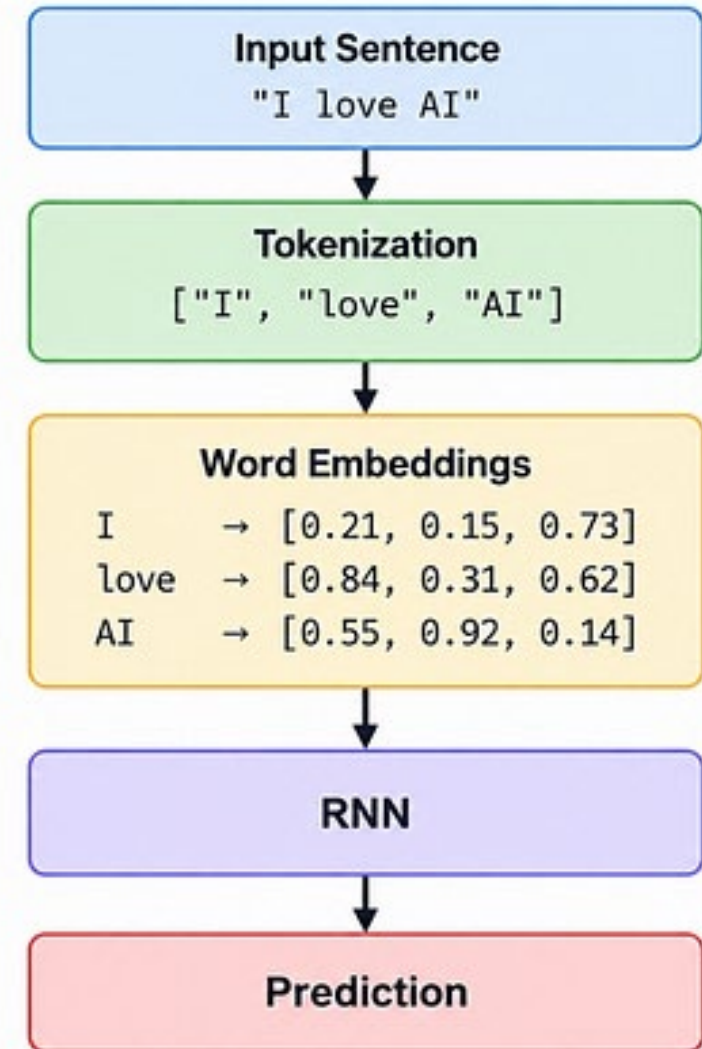
RNN for NLP (Natural Language Processing)

Before feeding text into an RNN, words need to be converted into numbers.

A common approach is to use word embeddings.

If input is number sequence, each number needs to be converted to a vector by input embeddings.

Embeddings represent number/words as numerical vectors, allowing the neural network to work with them mathematically.

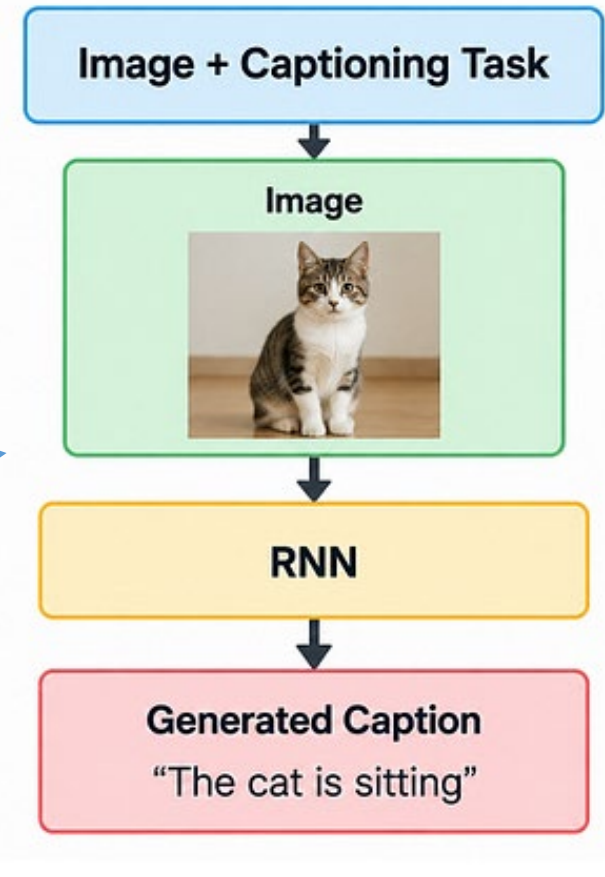


Different Types of RNN Architectures

RNNs can be used in different ways depending on the problem.

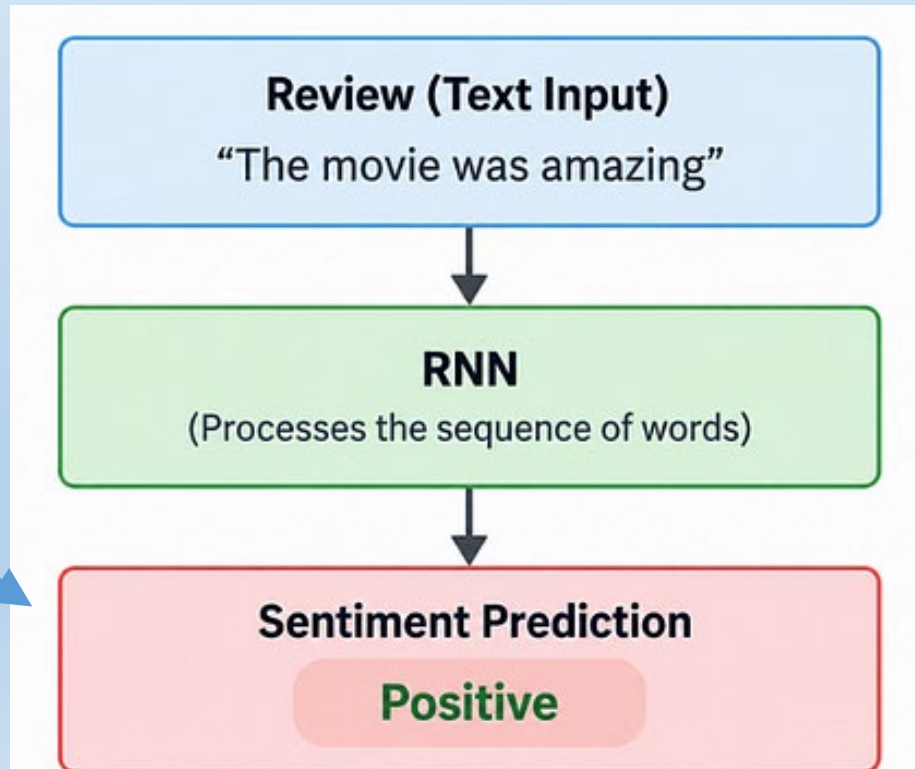
1. One-to-Many

One input produces a sequence of outputs.



2. Many-to-One

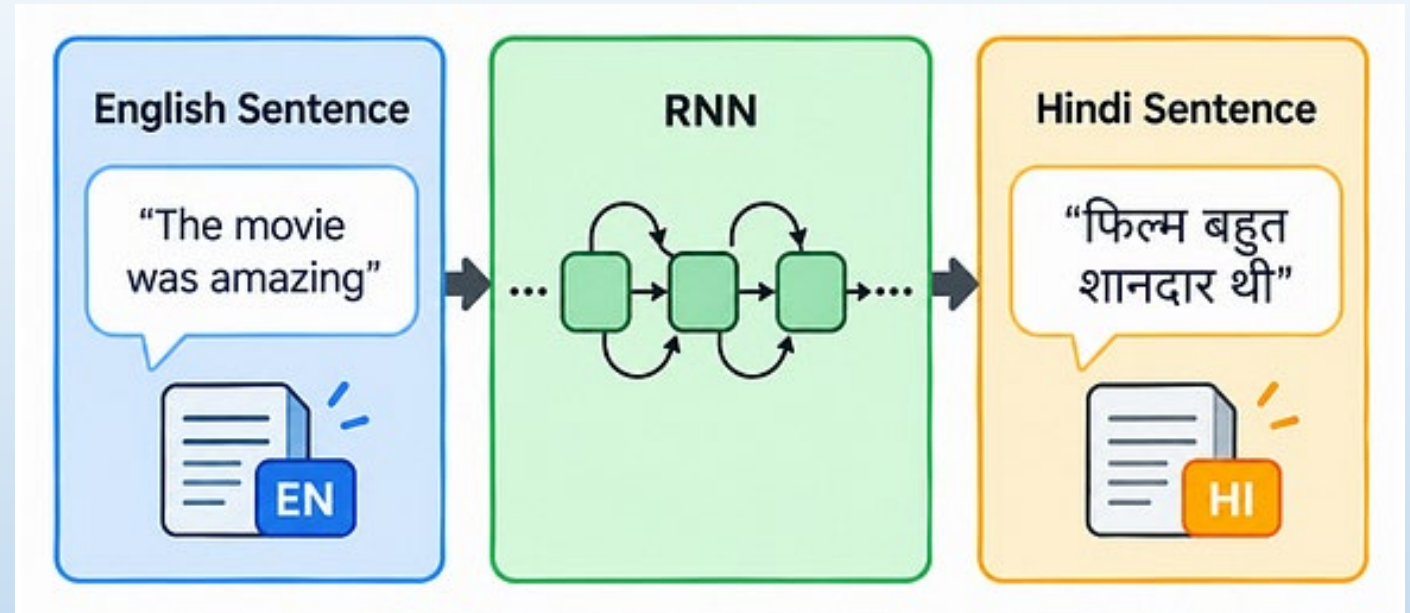
A sequence produces one output.



Different Types of RNN Architectures ... (cont'd)

3. Many-to-Many

A sequence produces another sequence.



The Problem with Basic RNNs

The main problem is that basic RNNs struggle with long-term dependencies. To understand some later words, the model may need information from much earlier in the sentence.

Basic RNNs often struggle to remember information over long sequences.

Basic Concepts

Differentiation: Gradients— especially the chain rule, which is used to understand how gradients move through a neural network.

Optimization: Gradient Descent to Adam — especially backpropagation and how gradients are used to update model parameters.

The Math of Neural Networks — forward propagation, backpropagation, activation functions, and weights.

The issue

“We sat near the river bank.”

“I deposited money in the bank.”

The model needs some way to carry information from earlier steps to later steps in order to understand the word “bank”.

The method: The Hidden State: A Running Summary

An RNN keeps a vector called the hidden state — think of it as a small memory, or a running summary of what the network has seen so far. At every step, the RNN: **Looks at the current input; Looks at the previous hidden state; Combines them; Produces a new hidden state.**

That new hidden state is then passed to the next step.

For example, imagine reading: “The cat sat”

"The" → update memory



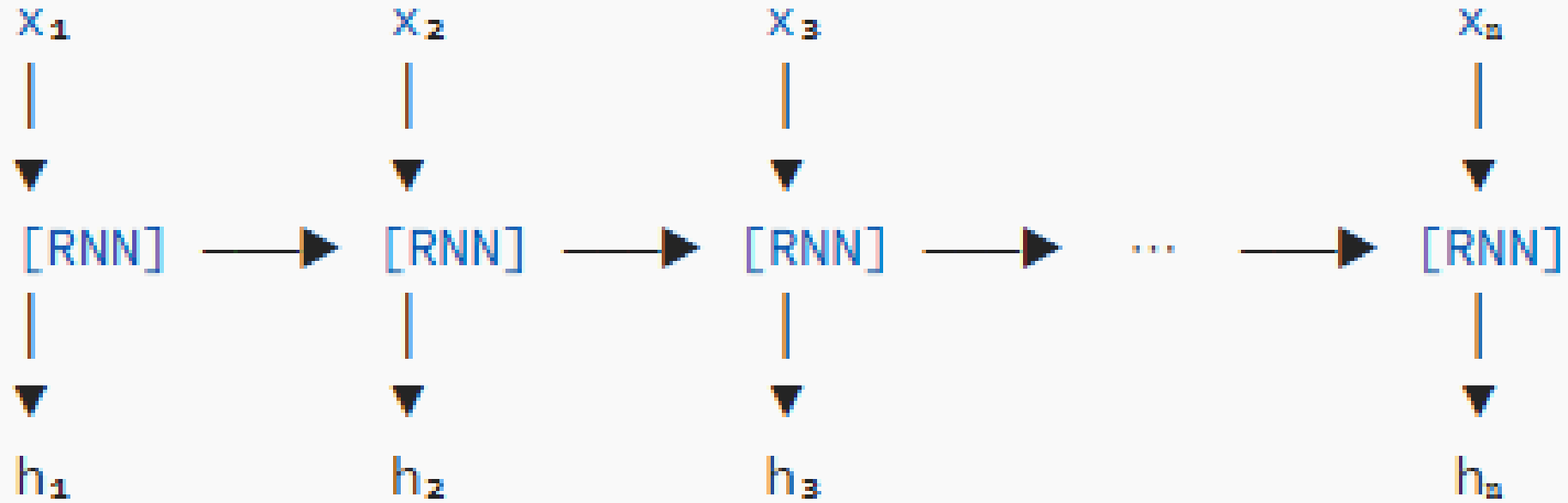
"cat" → update memory



"sat" → update memory

The hidden state does not literally store the sentence word-for-word — it is a vector of numbers that represents useful information learned by the network.

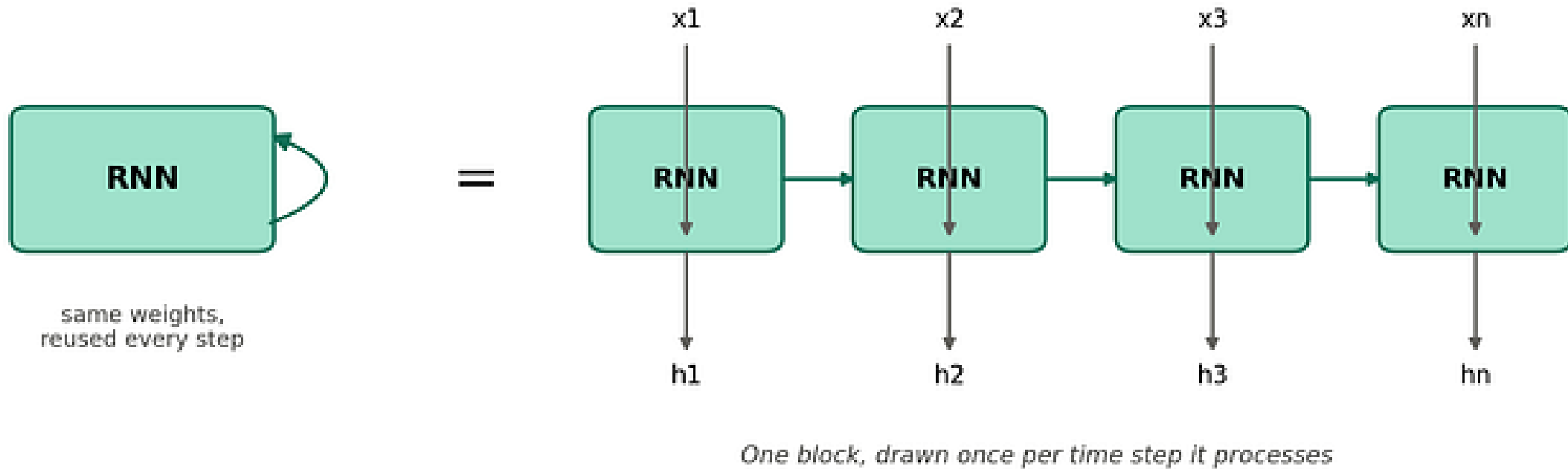
The RNN uses the same weights at every time step.



Here, $\mathbf{x}_t$ is the input at time step t , and $\mathbf{h}_t$ is the hidden state at time step t .

Every RNN block uses the same weights. The number of steps depends on the length of the sequence.

Unrolling the RNN loop



$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots$$

Each hidden state depends on the one before it

Why do gradients vanish ?

During training, the network uses backpropagation to figure out how its weights should change. Because an RNN processes many time steps, the error must be propagated backward through those time steps — this is called Backpropagation Through Time (BPTT).

At each step, the gradient is repeatedly multiplied by terms involving the RNN's weights and the derivative of the activation function. If a number is repeatedly multiplying by a number smaller than 1: 0.9^{50} approx. 0.005

Similar shrinking happens to the gradient as it travels backward through many time steps, the gradient can become extremely small — this is called the **vanishing gradient problem**.

When the gradient becomes almost zero, the earlier time steps receive almost no useful signal about how they should improve. As a result, the RNN struggles to learn relationships between things that are far apart in a sequence. (difficult to learn long-term dependencies)

The mathematics of RNN

At each time step, an RNN receives two things:

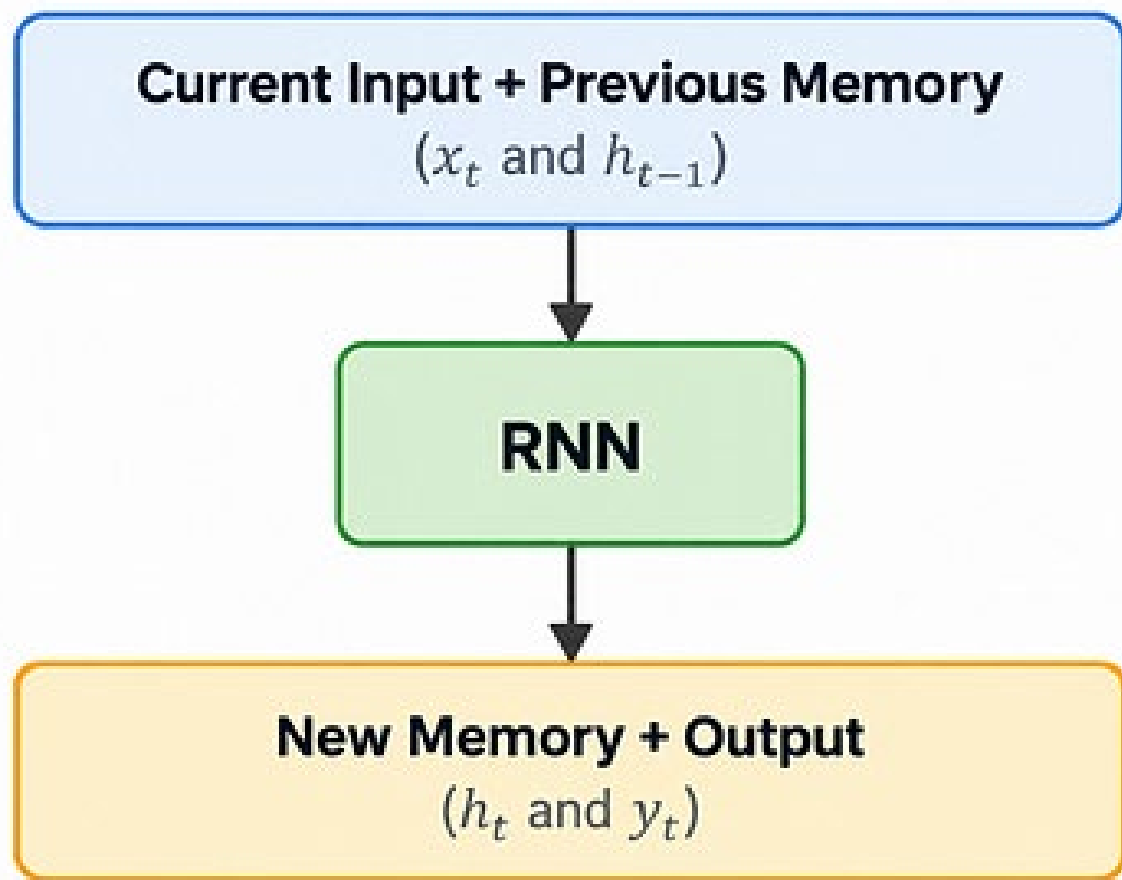
1. The current input x_t
2. The previous hidden state h_{t-1}

It combines them to produce a new hidden state h_t (the new memory).

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h) \text{ — the hidden-state equation}$$

Then, the RNN uses the new hidden state to produce an output:

$$y_t = W_o \cdot h_t + b_o \text{ — the output equation}$$



$$h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$$

x_t is current input

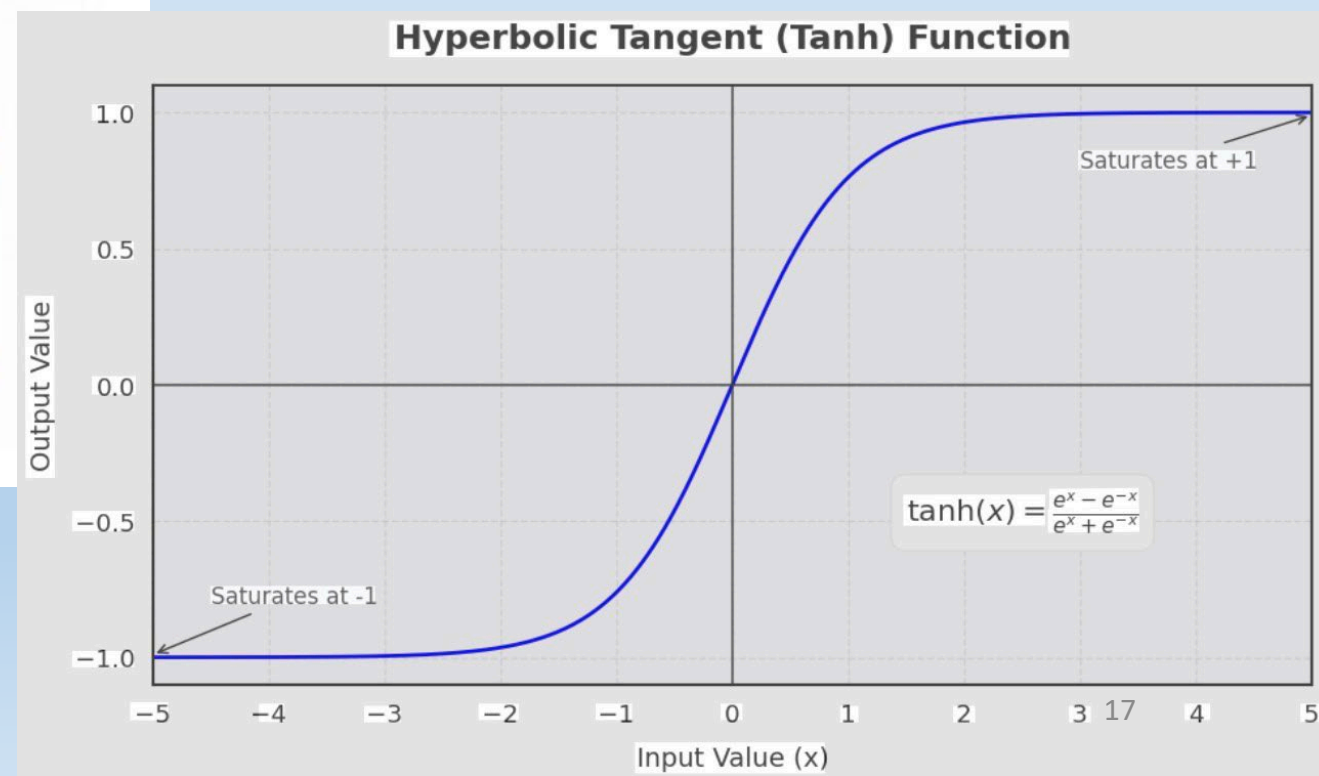
h_{t-1} is previous hidden state

h_t is current hidden state

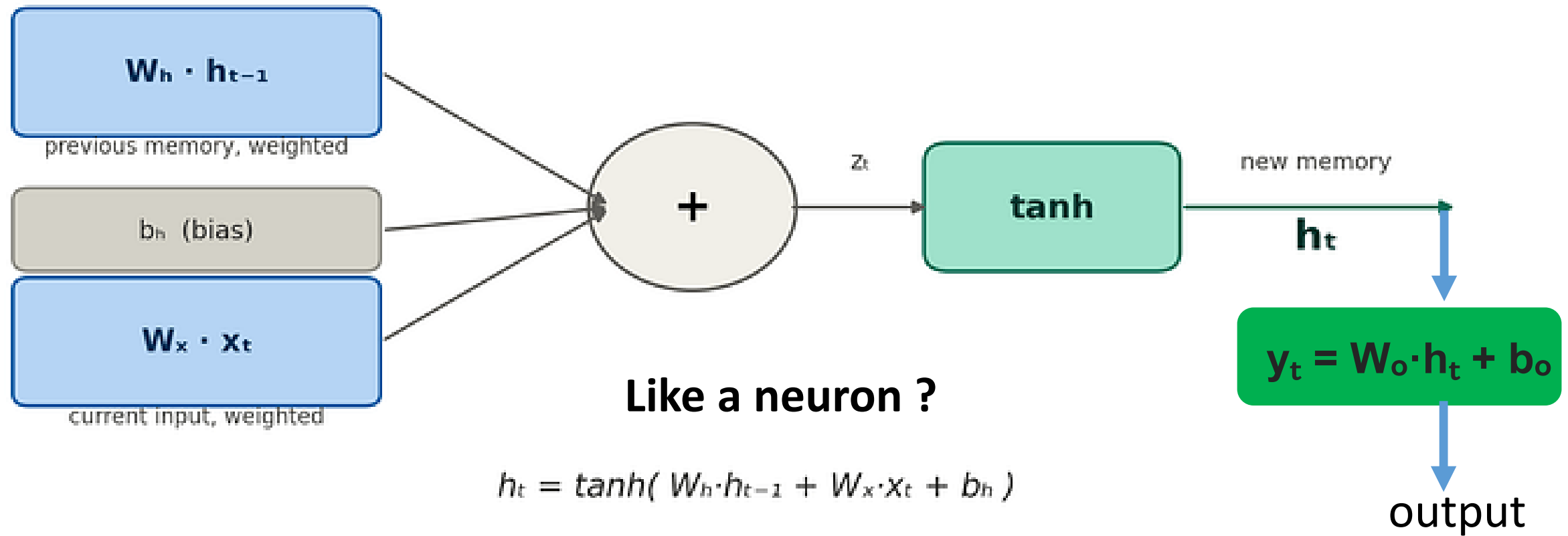
W_x, W_h are weights

b_h is bias

The activation function is $\tanh()$.



Updating the hidden state



Example: our sequence is: “The” → “cat” → “sat”

Step 1 — the RNN receives $x_1 = \text{“The”}$ and an initial hidden state h_0 .

$$h_1 = \tanh(W_h \cdot h_0 + W_x \cdot x_1 + b_h)$$

Step 2 — the next input is $x_2 = \text{“cat.”}$ But now the RNN also has h_1 , the memory from the previous step:

$$h_2 = \tanh(W_h \cdot h_1 + W_x \cdot x_2 + b_h)$$

Step 3 — the next input is $x_3 = \text{“sat.”}$ The previous memory is now h_2 :

$$h_3 = \tanh(W_h \cdot h_2 + W_x \cdot x_3 + b_h)$$

Note That The Same Weights(W_h , W_x) Are Reused. The RNN does **not** have a separate W_1 , W_2 , W_3 for different time steps. Instead, it has one set — **W_h , W_x , W_o** — and reuses them.

The same is true for the bias vectors **b_h** and **b_o** . This is the key idea behind the word “recurrent” — the network learns one set of rules and applies those rules repeatedly as it moves through the sequence.

$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow \dots$$

Each hidden state depends on the one before it.

What happens mathematically when an RNN learns?

The main challenge is that an RNN does not process all time steps independently. Each hidden state depends on the previous hidden state: $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$

So when the network makes an error at a later time step, that error may need to travel backward through many previous time steps. This is where Backpropagation Through Time (BPTT) comes in.

During the forward pass, information moves from left to right:

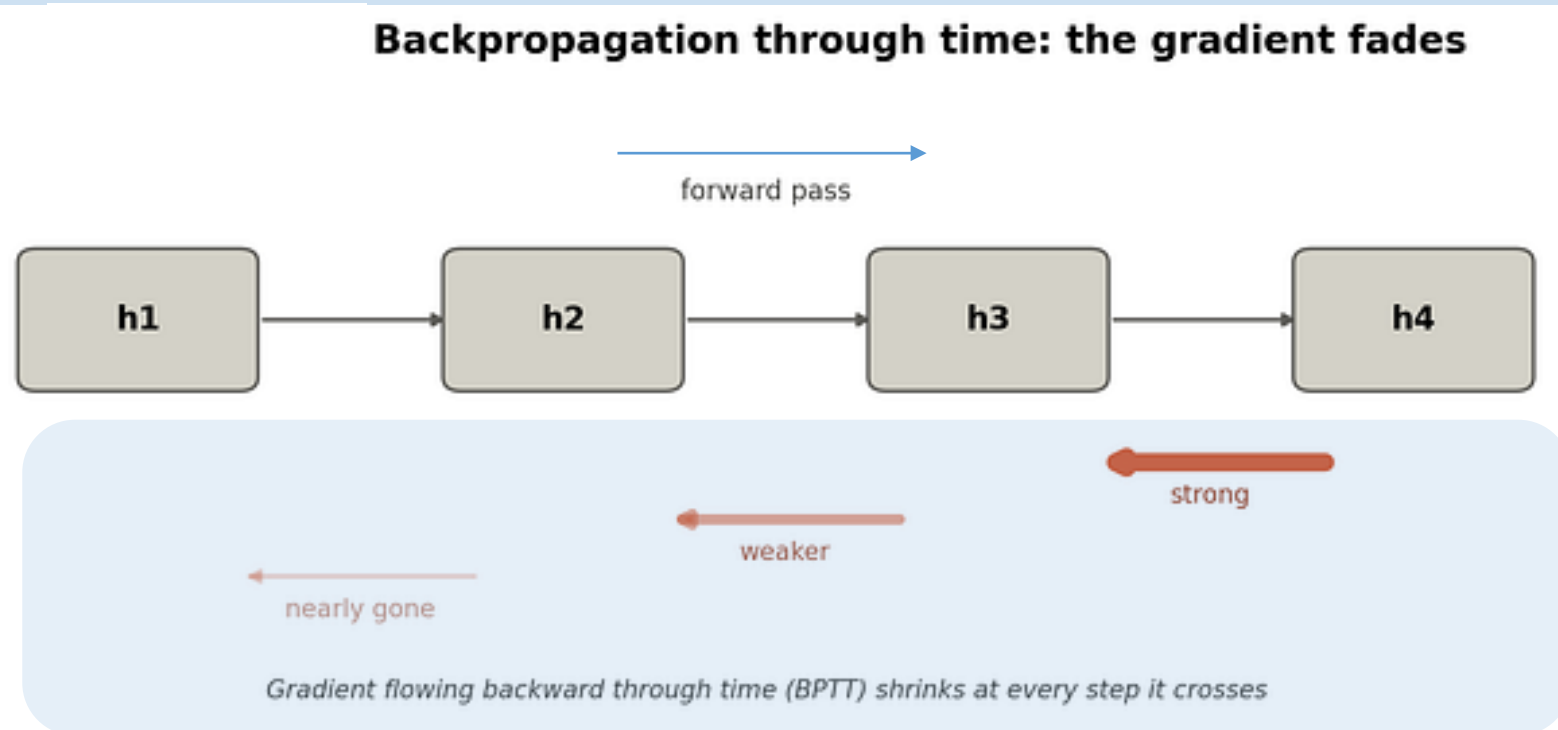
$$h_1 \rightarrow h_2 \rightarrow h_3 \rightarrow h_4$$

During backpropagation, the gradient moves in the opposite direction: $h_4 \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$

The chain rule appears. Suppose we want to know how much the final hidden state h_n depends on an earlier hidden state h_1 . The chain rule tells us that we multiply the effects across all the intermediate steps:

$$\partial h_n / \partial h_1 = (\partial h_n / \partial h_{n-1}) \cdot (\partial h_{n-1} / \partial h_{n-2}) \cdot \dots \cdot (\partial h_2 / \partial h_1)$$

This equation is the heart of the vanishing-gradient problem, because we're multiplying many terms together.



The Gradient Through One RNN Step

Remember the hidden-state equation: $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$

Focus only on how h_t depends on h_{t-1} . The transformation has two important parts: (1) Multiplication by W_h (2) Application of $\tanh$.

$$\begin{aligned} \partial h_n / \partial h_{n-1} &= d/dz [\tanh(z)] \cdot \partial [W_h \cdot h_{t-1} + W_x \cdot x_t + b_h] / \partial (h_{n-1}) \\ &= (1 - \tanh^2(z)) \cdot W_h \end{aligned}$$

Note that the derivative of the $\tanh$ function :

$$d/dz [\tanh(z)] = 1 - \tanh^2(z)$$

This derivative is never larger than 1. Therefore, the gradient passing through one RNN step depends on both W_h and $1 - \tanh^2(z_t)$, where z_t is the value entering the $\tanh$ function:

$$z_t = W_h \cdot h_{t-1} + W_x \cdot x_t + b_h$$

So, conceptually:

The tanh derivative is always between 0 and 1.

Gradient through one step $\approx W_h \times \tanh$ derivative

Now imagine that the gradient gets slightly smaller at every time step. The gradient has become almost zero. This is the basic idea behind the vanishing gradient problem — the gradient starts with useful information, but after passing through many time steps, it becomes extremely small.

When the gradient becomes very small, the early time steps receive almost no information about how they should change their weights. So the network has difficulty learning relationships between events that are far apart.

In simple words: the learning signal gets weaker and weaker as it travels backward through many time steps. That's the **vanishing gradient problem**.

Long sequence → many time steps → gradient must travel much farther → gradient may vanish or explode → learning becomes difficult

Vanilla RNNs have difficulty learning long-term dependencies — and it is not simply a problem of having too little data or too little computing power.

During training, RNNs use gradients to update their weights. Gradients are repeatedly passed backward through many time steps. Sometimes, the gradients become extremely small. This is due to the mathematics of repeatedly passing gradients through the recurrent computation.

This limitation of vanilla RNNs motivated the development of architectures designed to preserve information over much longer sequences → Long Short-Term Memory network (LSTM).

The **basic RNNs are not effective** when dealing with long sentences or long sequences.

LSTM and GRU: Better Versions of RNNs

LSTM: Long Short-Term Memory

LSTM uses **gates** to decide:

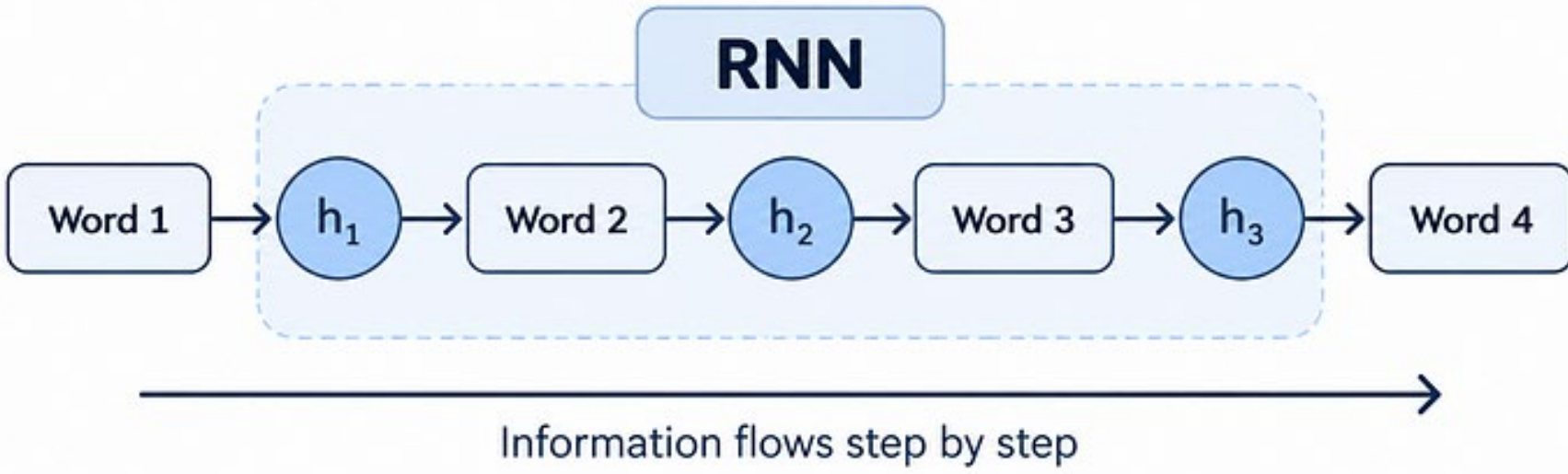
- What information to keep
- What information to forget
- What information to use

GRU: Gated Recurrent Unit

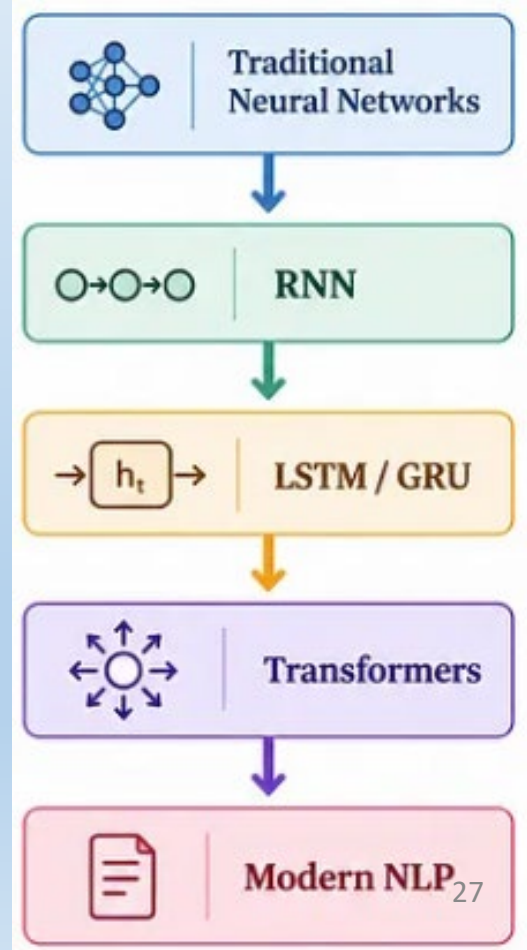
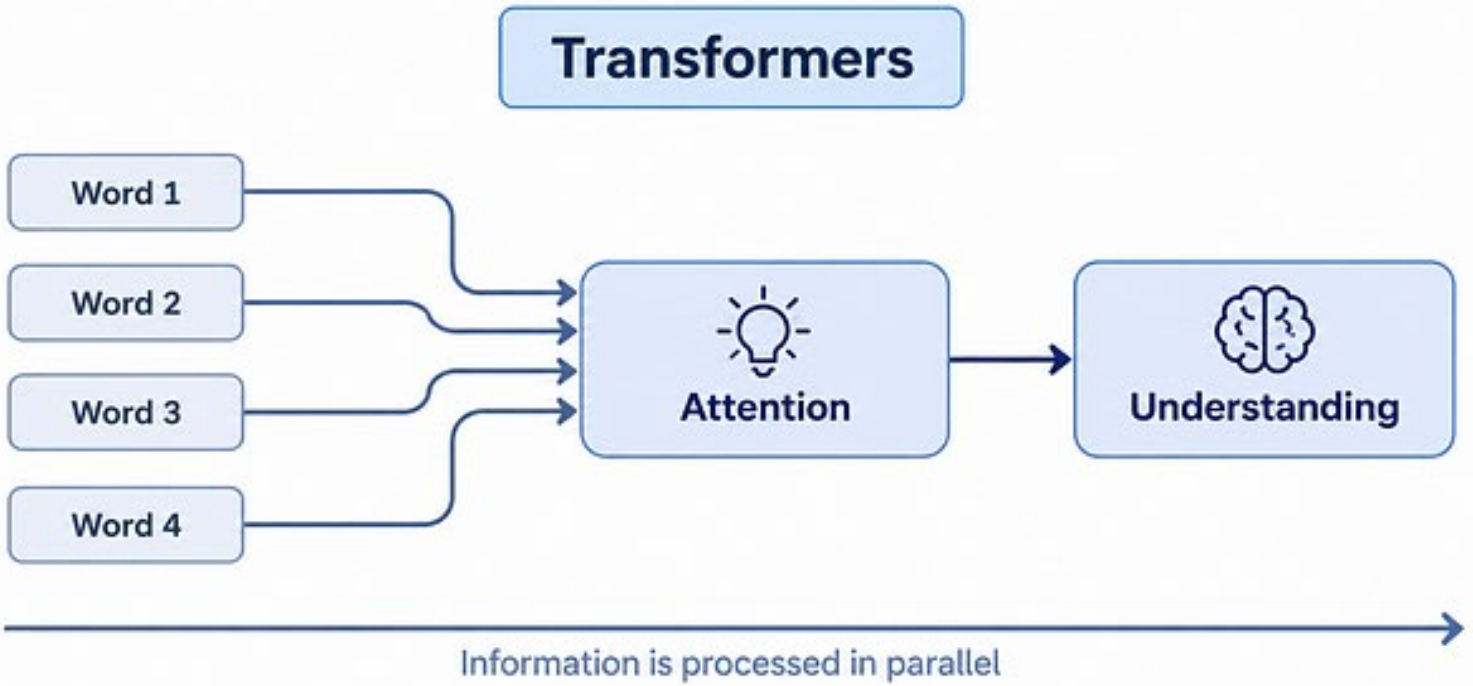
GRU works in a similar way but has a simpler architecture and fewer gates.

Understanding RNNs is important because they teach us the fundamental idea of learning from sequences and using previous information to understand current information. Yet, modern NLP relies heavily on Transformer models.

Models such as BERT and GPT use the Transformer architecture, which can process relationships between words more effectively and can be trained much more efficiently on large datasets.

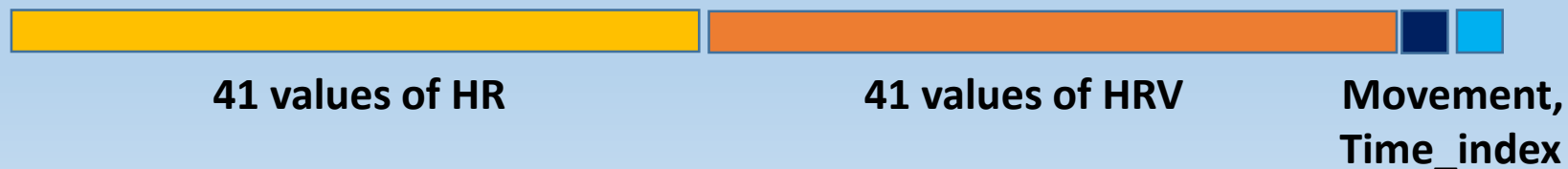
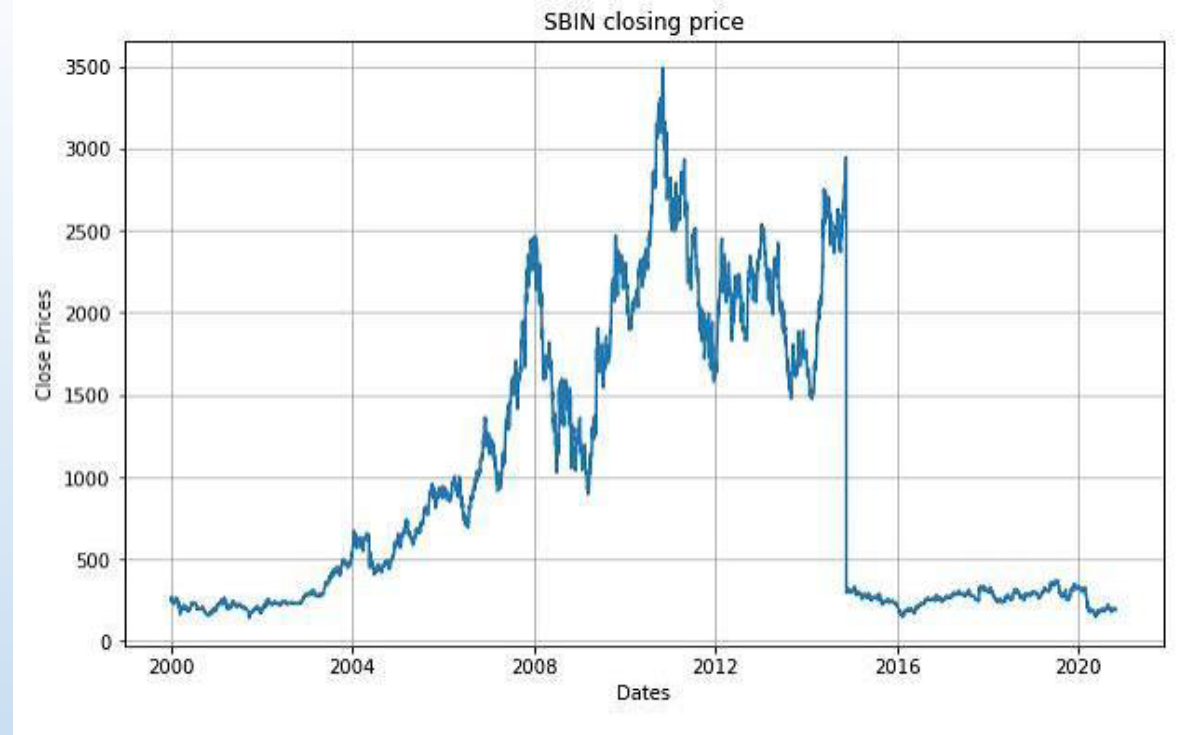


The key difference is that an RNN processes information sequentially, while Transformers use **attention** to look at relationships between different words more directly.



Example: Time series of Data

- Stock Price Prediction
- Sleep Stage Prediction based on HR series (past 20 mins), HRV series (past 20 mins), current Movement and time index.



Machine Learning Applications

- **Language modeling** — early neural language models used RNNs to predict the next word by carrying context forward through the hidden state.
- **Speech recognition** — audio is inherently sequential; RNNs (and later their gated variants) were a natural fit for processing frame-by-frame audio features.
- **Time-series forecasting** — stock prices, sensor readings, and demand forecasting all involve data where the order of observations carries meaning an RNN can exploit.
- **Sequence labeling** — tasks like part-of-speech tagging or named-entity recognition, where each element in a sequence needs a label that depends on its context.
- **Simple sequence generation** — character-level text generation, where the network predicts one character at a time based on everything generated so far.

Key Messages

- RNNs process a sequence one step at a time, reusing the same weights and passing a hidden state forward as a running memory.
- The hidden state update, $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$, is applied identically at every time step — this weight-sharing is what “recurrent” means.
- Training uses Backpropagation Through Time: the recurrent loop is unrolled into a chain, and ordinary backpropagation runs across it.
- Because the same weight matrix and tanh derivative are multiplied together at every step, gradients shrink exponentially with sequence length — the vanishing gradient problem.
- This single mathematical limitation is the direct **motivation for the LSTM**’s gated, memory design.

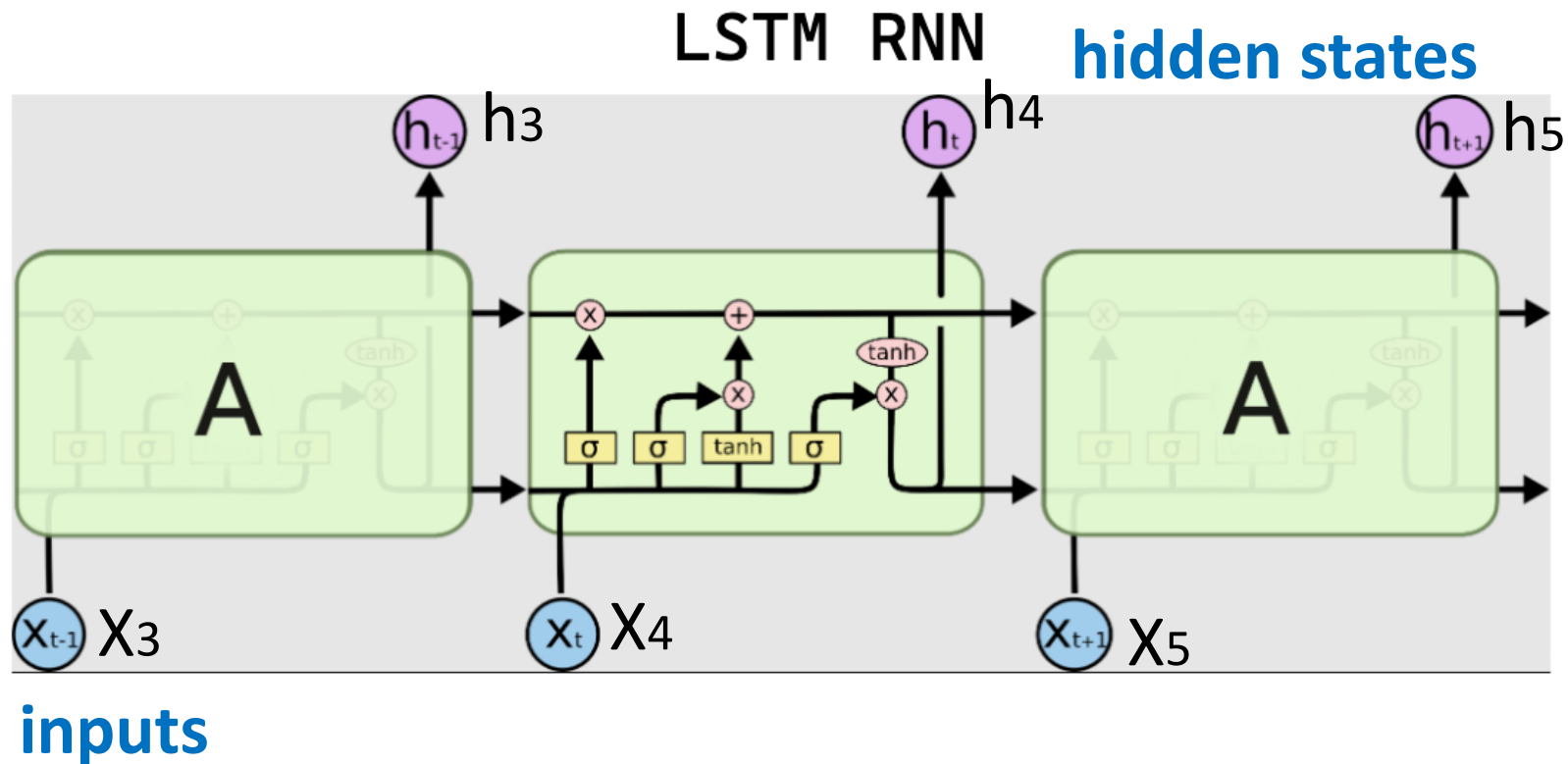
What is an LSTM ?

- The core concept behind the **Long Short-Term Memory (LSTM)** network is the regulation of information flow through **mathematical gates** to solve the vanishing gradient problem of standard Recurrent Neural Networks (RNNs).
- It introduces a persistent memory track called the **cell state**, which acts like a conveyor belt, allowing information to pass down the sequence largely unchanged unless modified by specific gated operations.
- The weight matrices (W) and bias vectors (b) are obtained through a process called **supervised training**, where the model learns the optimal values from data.
- Initially, these parameters are filled with random numbers, and they are iteratively refined using calculus and optimization algorithms.
- An LSTM network is trained by feeding it sequential data, making predictions, calculating the error, and using **Backpropagation Through Time (BPTT)** to update the weights.

The Forward-Pass from one input sequence X to get the output y .

An example of ten inputs of a time sequence values to an LSTM model.

The ten values form one input sequence $X = [x_1, x_2, \dots, x_{10}]$



$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$ forget gate

$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$ input gate

$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$
candidate cell gate

hidden state

h_t

new cell state

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

forget gate

f_t

i_t

$\tilde{C}_t$

o_t

output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

h_{t-1}

x_t

input

$$h_t = o_t * \tanh(C_t)$$

Many equations involved !

The h is a hidden state and c is a long-term cell state. Then, there are **three regulator structures called gates**, which are used to control the memory.

- Forget Gate f_t : Decides what old information to throw away from the cell state.
- Input Gate i_t : Decides what new information to store in the cell state.
- Output Gate o_t : Decides what parts of the updated cell state to show as the new hidden state.
- **Note that the weight matrices and biases are the same in each block.**

The complete recurrence
at time t:

$$z_t = [x_t, h_{t-1}]$$

$$f_t = \sigma(W_f z_t + b_f)$$

$$i_t = \sigma(W_i z_t + b_i)$$

$$\tilde{c}_t = \tanh(W_c z_t + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o z_t + b_o)$$

$$h_t = o_t \odot \tanh(c_t)$$

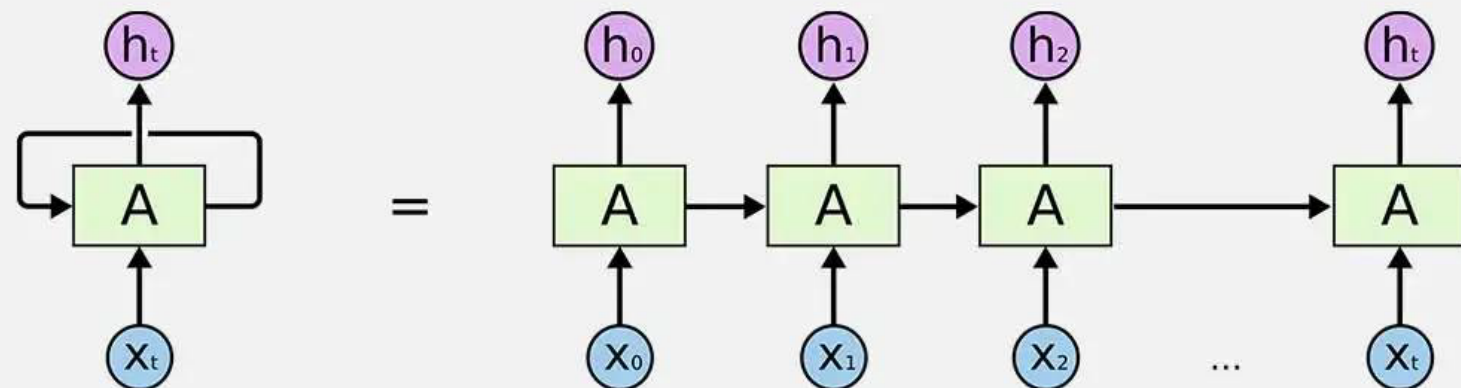
This process is repeated for every time step:

$$(x_1, h_0, c_0) \rightarrow (h_1, c_1)$$

$$(x_2, h_1, c_1) \rightarrow (h_2, c_2)$$

...

$$(x_T, h_{T-1}, c_{T-1}) \rightarrow (h_T, c_T)$$



Training an LSTM

The Forward Pass

- a) Initialize the hidden state (h_0) and cell state (C_0) to vectors of zeros.
- b) Pass the input features of the sequence step-by-step through the network.
- c) Compute the gated operations (f_t , i_t , o_t) at each time step.
- d) Generate the final output ($\hat{y}$) from the last hidden state of the sequence.

Calculate the Prediction Loss

- a) Compare the model's output ($\hat{y}$) against the actual ground-truth label (y) using a loss function.
- b) For predicting a category: Use Cross-Entropy Loss.
- c) For predicting a continuous number: Use Mean Squared Error (MSE).

Run Backpropagation Through Time (BPTT)

- a) Calculate the gradient of the loss starting from the final time step.
- b) Unroll the network backward through time, accumulating gradients for the weights (W) and biases (b) at every single sequence step.
- c) Apply Gradient Clipping: If gradients get too large during unrolling, clip them to a maximum threshold (e.g., 1.0) to prevent numerical explosion.

Update the Weight Matrices

The optimization algorithm (usually Adam or RMSprop) updates the internal matrices based on the accumulated gradients.

$$W_{\text{new}} = W_{\text{old}} - \text{learning rate} \times \text{gradient}$$

$$W_{t+1} = W_t - \alpha \cdot \frac{\partial L}{\partial W_t}$$

$$b_{t+1} = b_t - \alpha \cdot \frac{\partial L}{\partial b_t}$$

Conclusions

- Long Short-Term Memory (LSTM) networks are a powerful type of recurrent neural network designed to handle sequential data by overcoming traditional vanishing gradient problems.
- Their gating mechanisms (forget, input, and output gates) allow them to retain or discard information dynamically over long sequences.
- Weakness: LSTMs process data one time step at a time, preventing parallelization during training and slowing down computation for large systems.
- Weakness: The complex gating architecture makes them computationally expensive and memory-intensive.

--- The End ---